

Операционные системы

Лекция 17

Операции над файлами

План лекции

- Файловые операции и способы их организации
- Открытие файла, обмен данными с файлом, блокировки файлов
- Стандартные файлы ввода-вывода, перенаправление вывода

ФС с запоминанием и без запоминания состояния операций

Файловая система ОС предоставляет пользователям набор операций для работы с файлами, оформленный в виде системных вызовов, таких, например, как ***creat*** (создать файл)

Чаще всего с одним и тем же файлом пользователь выполняет не одну операцию, а последовательность операций. Например, при работе текстового редактора с файлом, в котором содержится некоторый документ, пользователь обычно считывает несколько страниц текста, редактирует эти данные и записывает их на место считанных, а затем считывает страницы из другой области файла и т. п.

После большого количества операций чтения и записи пользователь завершает работу с данным файлом и переходит к другому

ФС с запоминанием и без запоминания состояния операций

Какие бы операции над файлом ни выполнялись операционной системой, ***read*** или ***write, creat*** или ***delete***, они непременно включают ряд универсальных действий, повторяемых в неизменном виде при выполнении любой операции

1. По символьному имени файла ищутся его характеристики, которые хранятся в файловой системе на диске
2. Характеристики файла копируются в оперативную память, так как только таким образом программный код может их использовать
3. На основании характеристик файла проверяются права пользователя на выполнение запрошенной операции (чтение, запись, удаление, просмотр атрибутов файла)
4. Очищается область памяти, отведенная под временное хранение характеристик файла

ФС с запоминанием и без запоминания состояния операций

Помимо таких универсальных действий, каждая операция с файлом включает ряд *специфических* для этой операции действий

Например, считывание данных файла присуще только операции *read* (читать из файла), а запись новых данных в файл происходит только при выполнении операции *write* (записать в файл)

Пусть имеется некая последовательность операций с одним и тем же файлом X, например: read(X), write(X), ...

Операционная система может выполнить эту последовательность двумя способами

ФС с запоминанием и без запоминания состояния операций

Первый подход состоит в последовательном выполнении для каждой операции всех относящихся к ней действий, как универсальных, так и специфических (а)

То есть когда из прикладной программы поступает системный вызов на выполнение операции `read(X)`, ФС ее выполняет, отсылает ответ, а затем удаляет из своих внутренних таблиц всю информацию о выполненной операции и «с чистого листа» приступает к выполнению операции `write(X)`

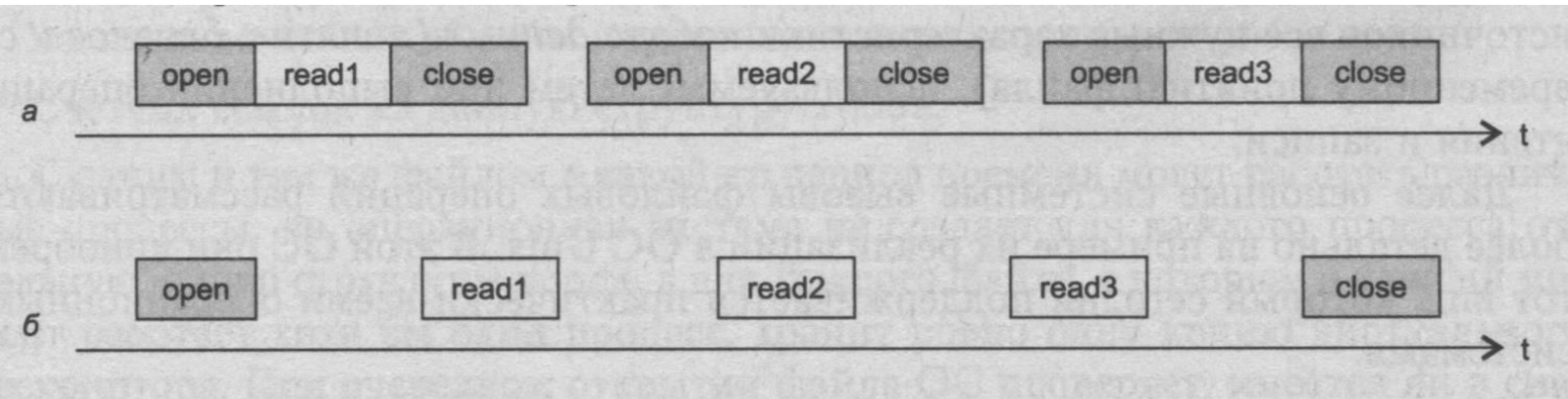
ФС с запоминанием и без запоминания состояния операций

В соответствии со вторым подходом универсальные действия выполняются лишь в начале и в конце последовательности операций, а для каждой промежуточной операции выполняются только специфические действия (б)

То есть если первой была выполнена операция чтения `read(X)`, то при выполнении следующей операции `write (X)` вовсе не требуется снова производить поиск и перенос в оперативную память адреса файла и других его характеристик

Действительно, эта информация уже была найдена во время выполнения предыдущей операции `read (X )`, достаточно просто ее сохранить для повторного использования

ФС с запоминанием и без запоминания состояния операций



ФС с запоминанием и без запоминания состояния операций

В первом случае файловую систему относят к типу **ФС без запоминания состояния операций** (stateless). ФС такого типа не хранит никакой информации о предыдущих операциях, все операции выполняются абсолютно независимо друг от друга

Во втором случае ФС запоминает информацию, относящуюся к универсальным действиям, полученную ею при выполнении самой первой операции с тем, чтобы использовать ее для выполнения всех последующих операций с файлом, вплоть до самой последней. Такие ФС относят к типу **ФС с запоминанием состояния операций** (stateful)

ФС с запоминанием и без запоминания состояния операций

Преимуществом первого способа является его большая устойчивость к сбоям в работе системы, так как каждая операция является самодостаточной и не зависит от результата предыдущей

Поэтому первый способ иногда применяется в распределенных сетевых файловых системах (например, в Network File System, NFS компании Sun), когда сбои из-за потерь пакетов или отказов одного из сетевых узлов более вероятны, чем при локальном доступе к файлам

Однако большинство файловых систем поддерживает второй способ организации, при котором файловая система сохраняет информацию о предыдущих операциях, как более экономичный и быстрый

ФС с запоминанием и без запоминания состояния операций

При втором способе в файловой системе вводится два специальных системных вызова: **open** (открытие файла) и **close** (заккрытие файла)

Системный вызов открытия файла **open** выполняется перед началом любой последовательности операций с файлом, а вызов закрытия файла **close** – после окончания работы с файлом

Основной задачей вызова **open** является преобразование символьного имени файла в его уникальное числовое имя, копирование характеристик файла из дисковой области в буфер оперативной памяти и проверка прав пользователя на выполнение запрошенной операции

Вызов **close** освобождает буфер с характеристиками файла и делает невозможным продолжение операций с файлом без его повторного открытия

Открытие файла

Системный вызов **open()** в ОС Unix работает с двумя аргументами:

- Символьным именем открываемого файла
- Параметром, задающим режим открытия файла

Режим открытия говорит системе, какие операции будут выполняться с файлом в последовательности операций до закрытия файла по системному вызову **close**, например, только чтение, только запись или чтение и запись

Открытие файла

При открытии файла ОС сначала выполняет преобразование первого аргумента системного вызова, то есть символьного имени файла, в его уникальное числовое имя, которым в традиционных файловых системах Unix является номер индексного дескриптора

По номеру индексного дескриптора inode файловая система находит нужную запись на диске и копирует из нее характеристики файла в оперативную память

Открытие файла

Для хранения копии индексного дескриптора используются буферные области системного виртуального пространства

Характеристики индексного дескриптора, перенесенные в оперативную память, помещаются в структуру так называемого **виртуального дескриптора** vnode (virtual node)

Структура vnode включает поля индексного дескриптора файла inode, а также несколько перечисленных далее дополнительных полей, полезных при выполнении операций с файлом

Открытие файла

- Состояние индексного дескриптора в памяти, отражающее:
 - заблокирован ли файл
 - ждет ли снятия блокировки с файла какой-либо процесс
 - отличается ли представление характеристик файла в памяти от своей дисковой копии в результате изменения содержимого индексного дескриптора
 - отличается ли представление файла в памяти от своей дисковой копии в результате изменения содержимого файла
 - является ли файл точкой монтирования
- Логический номер устройства файловой системы, содержащей файл
- Номер индексного дескриптора. В дисковом индексном дескрипторе это поле отсутствует, так как номер определяется положением дескриптора относительно начала области индексных дескрипторов
- Счетчик ссылок на данную структуру vnode

Открытие файла

С одним и тем же файлом в какой-то период времени могут работать различные процессы, но операционная система не создает для каждого процесса отдельную копию структуры vnode, а для каждого файла, с которым в данный момент работает хотя бы один процесс, хранит ровно одну копию виртуального дескриптора

При очередном открытии файла ОС проверяет, имеется ли в системной памяти структура vnode открываемого файла (по номеру логического устройства и номеру индексного дескриптора, которые определяются при преобразовании символьного имени), и если имеется, то счетчик ссылок на нее увеличивается на единицу

При очередном закрытии этого файла счетчик ссылок уменьшается на единицу, и если он становится равным 0, то буфер, хранящий данную структуру vnode, считается свободным.

Открытие файла

Использование единственной копии характеристик файла и некоторых характеристик файловых операций (например, признака блокировки), общих для всех работающих с файлом процессов, экономит системную память

Тем не менее существуют характеристики, индивидуальные для каждого процесса, выполняющего некоторую последовательность операций с определенным файлом

Для их хранения в Unix используется структура типа **file**, которая так же, как и `vnode`, хранится в системной области памяти

Открытие файла

При каждом открытии процессом файла ОС проверяет права пользовательского процесса на выполнение запрошенной операции с файлом, и если проверка прошла успешно, создает в системной области памяти новую структуру **file**, описывающую как сам открытый файл, так и операции, которые процесс собирается производить с файлом (например, чтение)

Открытие файла

Структура **file** содержит следующие поля:

- Признак режима открытия (только для чтения, для чтения и записи и т. п.)
- Указатель на структуру vnode
- Текущее смещение в файле (переменная offset) при операциях чтения/записи
- Счетчик ссылок на данную структуру
- Указатель на структуру, содержащую права процесса, открывшего файл (эта структура находится в дескрипторе процесса)
- Указатели на предыдущую и последующую структуру file, связывающие все такие структуры в двойной список

Открытие файла

Переменная `offset`, хранящаяся в структуре `file`, позволяет ОС запомнить текущее положение условного указателя в последовательности байтов файла

При открытии файла эта переменная указывает на начальный или конечный байт файла в зависимости от заданного режима открытия

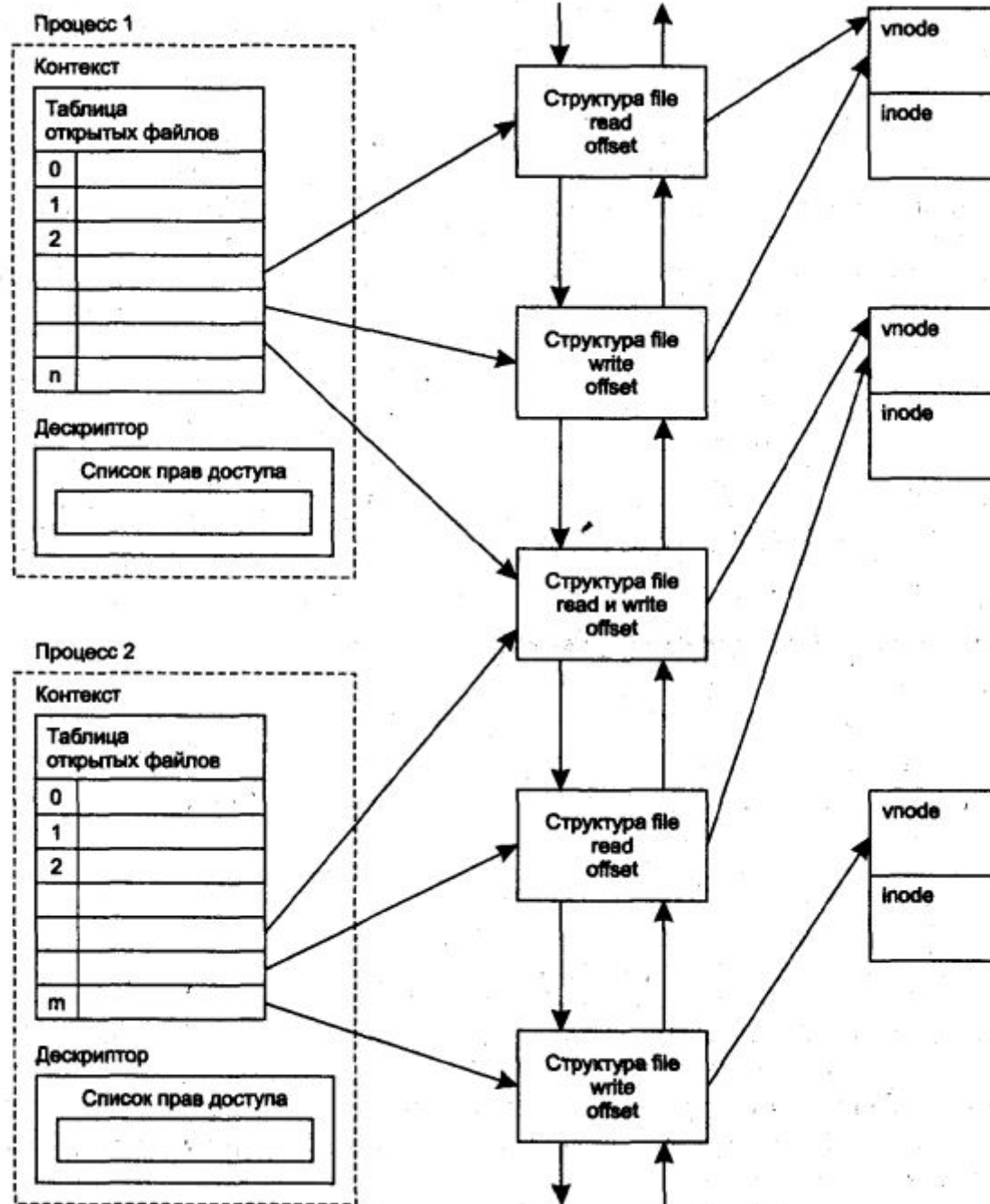
После выполнения операций чтения или записи указатель сдвигается на количество байтов, прочитанное или записанное в результате операции

Следующая операция заставит указатель в том состоянии, в котором его оставила предыдущая операция

Прикладной программист может явно управлять положением указателя с помощью системного вызова `lseek`

Открытие файла

При каждом новом открытии какого-либо файла ОС создает новую структуру file и помещает ее в дважды связанный список. Обычно под хранение структур file в системной области отводится ограниченная область, поэтому общее количество открытых файлов всеми процессами в любой момент ограничено



Открытие файла

После создания структуры `file` операционная система помещает указатель на нее в таблицу открытых файлов процесса, которая находится в контексте процесса

Если процесс несколько раз открывает один и тот же файл, то структура `file` создается для каждой операции открытия. Так как контекст процесса-родителя в Unix наследуется процессом-потомком, потомок наследует и указатели на все открытые родителем файлы, получая возможность выполнять с ними операции

Открытие файла

Системный вызов **open** возвращает в пользовательский процесс **дескриптор файла**, который представляет собой номер записи в таблице открытых файлов процесса

Дескриптор файла имеет локальное значение только для того процесса, который открыл файл, для разных процессов одно и то же значение дескриптора указывает на разные операции в общем случае с разными файлами

После открытия файла его дескриптор используется во всех дальнейших операциях с файлом вплоть до явного закрытия файла

Таким образом, дескриптор файла является временным уникальным именем, но не файла, а конкретной последовательности операций с этим файлом, ограниченной командами **open** и **close**

Открытие файла

Для открытия файла `/bin/progl.exe` в режиме «только для чтения» прикладной программист может использовать следующее выражение на языке C:

```
fd = open("/bin/progl.exe", O_RDONLY);
```

Здесь `fd` – это целочисленная переменная, сохраняющая значение дескриптора открытого файла. Ее значение должно использоваться в операциях обмена данными с файлом `/bin/progl.exe`

При неудачной попытке открытия файла (нет прав для выполнения затребованной операции, неверное имя файла) переменной `fd` присваивается значение `-1`, которое является индикатором ошибки для всех системных вызовов Unix

Обмен данными с файлом

Для обмена данными с предварительно открытым файлом в ОС Unix существуют системные вызовы **read** и **write**

В том случае, когда необходима явным образом указать, с какого байта файла читать или записывать данные, используется также системный вызов **lseek**

Системный вызов чтения данных из файла `read` имеет три аргумента:

```
read(fd, buffer, nbytes);
```

Обмен данными с файлом

Первый аргумент `f d` является целочисленной переменной, имеющей значение дескриптора открытого файла

Второй аргумент `buffer` является указателем на область пользовательской памяти, в которую система должна поместить считанные данные. Количество байтов этой области памяти задается третьим целочисленным аргументом `nbytes`

Функция `read` возвращает действительное количество считанных байтов (оно может отличаться от заданного, если, например, была указана область чтения, выходящая за пределы файла) или же код ошибки `-1`

Обмен данными с файлом

Начало дисковой области, которую нужно прочитать с помощью вызова `read`, явно в этом системном вызове не указывается

Чтение начинается с того байта, который задается смещением `offset` в структуре `file`

На это смещение указывает запись с номером `fd` в таблице открытых файлов процесса

После выполнения вызова `read` смещение `offset` наращивается на количество считанных байтов

Обмен данными с файлом

Системный вызов записи данных `write` аналогичен вызову `read`:

```
write(fd, buffer, nbytes);
```

Функция `write` записывает количество байтов `nbytes` из буфера оперативной памяти `buffer` в файл, описываемый дескриптором `fd`

Функция `write`, так же как и `read`, возвращает вызвавшей ее программе значение реально переданных ею байтов или код ошибки

Обмен данными с файлом

Рассмотрим пример, в котором прикладная программа работает с файлом, состоящем из записей фиксированной длины по 50 байт:

```
fd = open('/doc/qwery/basel2.txt', O_RDWR);
```

```
read(fd, buffer1, 50);
```

```
read(fd, buffer2, 2500);
```

```
.....
```

```
lseek(fd, 150, 0);
```

```
write (fd, output, 300);
```

```
close(fd);
```

Обмен данными с файлом

В приведенном фрагменте программы после открытия файла `/doc/query/base12.txt` для чтения и записи выполняется чтение первой записи файла, а затем читается область файла, включающая еще 50 записей, со 2 по 51

После обработки считанных записей (эти инструкции опущены) указатель смещения в файле переводится на начало четвертой записи, и выполняется запись результатов в шесть последовательных записей, начиная с четвертой

Завершается фрагмент закрытием файла с помощью системного вызова `close`

Обмен данными с файлом

Все описанные системные вызовы являются синхронными, то есть пользовательский процесс переводится в состояние ожидания до тех пор, пока операция ввода-вывода не завершится

Описанный набор системных вызовов, появившийся в ОС Unix еще в 70-х годах, стал стандартом де-факто для современных операционных систем. Эти традиционные системные вызовы часто в конкретных ОС дополняются оригинальными системными вызовами ввода-вывода, например операциями асинхронного типа

На основе системных вызовов ввода-вывода обычно строятся более мощные библиотечные функции ввода-вывода, составляющие прикладной интерфейс ОС

Блокировки файлов

Блокировки файлов и отдельных записей в файлах являются средством синхронизации работающих в кооперации процессов, пытающихся использовать один и тот же файл одновременно

Процессы могут иметь соответствующие права доступа к файлу, но одновременное применение этих прав (в особенности права записи) может привести к некорректным результатам

Примером такой ситуации является одновременное редактирование одного и того же документа несколькими пользователями

Если доступ к файлу не управляется блокировками, то каждый пользователь, который имеет право записи в файл, работает со своей копией данных файла

Результат такого редактирования непредсказуем – он зависит от того, в какой последовательности записывали изменения в файл применяемые пользователями приложения-редакторы

Блокировки файлов

Многопользовательские операционные системы обычно поддерживают специальный системный вызов, позволяющий программисту устанавливать и проверять блокировки на файл и его отдельные области

В Unix такой системный вызов называется `fcntl`. В его аргументах указываются дескриптор файла, для которого нужно установить или проверить блокировки, тип операции (блокирование или проверка, блокирование доступа для чтения или для записи), а также область блокирования – смещение от начала файла и размер в байтах

Блокировки файлов

При проверке наличия блокировок, установленных другими процессами, вызов `fcntl` немедленно возвращает управление с сообщением результата

При установке блокировки можно задать два режима работы системного вызова:

- С переходом процесса в состояние ожидания в том случае, если блокировку установить невозможно (синхронный системный вызов)
- С немедленным возвратом в такой ситуации с сообщением отрицательного результата (асинхронный вызов)

Блокировки файлов

Запрошенная блокировка записи не может быть установлена в том случае, если другой процесс уже установил свою блокировку записи на тот же файл

То есть блокировка записи является ***исключительной***

Блокировки чтения не являются исключительными и могут устанавливаться на файл в том случае, если их области действия не перекрываются

Если на какую-то область файла установлена блокировка чтения, то на эту область нельзя установить блокировку записи

Блокировки файлов

В Unix существует два режима действия блокировок – **консультативный** (advisory) и **обязательный** (mandatory)

Основным рекомендуемым для использования режимом является консультативный. При нем операционная система не занимается блокированием операций с файлом, а только устанавливает признаки блокирования областей в структурах file, поддерживающих операции с файлами

Кооперирующиеся процессы обязательно должны проверять наличие блокировок на файл, чтобы синхронизировать свою работу. Если же блокировки установлены, но процесс не проверяет их, то операционная система не запрещает доступ процесса к файлу, когда процесс делает системные вызовы **read** или **write**

Стандартные файлы ввода и вывода, перенаправление вывода

В ОС Unix были введены в свое время понятия стандартных файлов:

- Стандартный файл ввода
- Стандартный файл вывода
- Стандартный файл ошибок

Эти три уже открытых файла существуют у любого пользовательского процесса с момента его возникновения

Процесс в любое время может организовать ввод данных из стандартного файла ввода, выполнив следующий системный вызов: `read(stdio, bufer, nbytes)`

Стандартные файлы ввода и вывода, перенаправление вывода

Здесь `stdio` – предопределённое имя константы, обозначающей дескриптор стандартного файла ввода

Аналогично, так как `stdout` – предопределенное имя дескриптора стандартного файла вывода, процесс может вывести данные в стандартный файл вывода, применив следующий системный вызов: `write(stdout, buffer, nbytes)`

За стандартным файлом ошибок закреплено имя `stderr`

Стандартные файлы ввода и вывода, перенаправление вывода

Фактически, при создании нового процесса ОС помещает в его таблицу открытых файлов три записи: с номером 0 – для стандартного файла ввода (следовательно, `stdin` всегда имеет значение 0), с номером 1 – для стандартного файла вывода (`stdout` = 1) и с номером 2 – для стандартного файла ошибок (`stderr` = 2)

Соответственно, создается и три структуры типа `file`, на которые указывают первые три записи таблицы открытых файлов процесса

Стандартные файлы ввода и вывода, перенаправление вывода

В начальный период существования процесса эти три структуры file связываются операционной системой с одним файлом

В качестве этого файла выступает специальный файл – терминал, с которого вошел в систему пользователь

Такое назначение стандартных файлов достаточно естественно. Прикладные программы, запускаемые пользователем в ходе сеанса работы, чаще всего выводят результаты и сообщения об ошибках на экран терминала, за которым работает пользователь, и с клавиатуры этого же терминала считывают команды и другие исходные данные

Управление файлами – Создание\Открытие

Windows API

```
HANDLE CreateFile(  
    LPCTSTR lpFileName,           // имя файла  
    DWORD dwDesiredAccess,        // способ доступа  
    DWORD dwShareMode,            // режимы совместного использования  
    LPSECURITY_ATTRIBUTES lpSecurityAttributes, // атрибуты защиты  
    DWORD dwCreationDisposition,  // создание или открытие файла  
    DWORD dwFlagsAndAttributes,   // флаги и атрибуты  
    HANDLE hTemplateFile           // файл атрибутов  
);
```

POSIX API

```
[OH] #include <sys/stat.h>  
#include <fcntl.h>  
  
int open(const char *path, int oflag, ...);  
int openat(int fd, const char *path, int oflag, ...);
```

Управление файлами – Заккрытие

Windows API

```
BOOL  CloseHandle(  
    HANDLE  hObject    // дескриптор объекта  
);
```

POSIX API

```
#include <unistd.h>  
  
int close(int fildes);  
int posix_close(int fildes, int flag);
```

Управление файлами – Чтение

Windows API

```
BOOL ReadFile(  
    HANDLE    hFile,                // дескриптор файла  
    LPVOID    lpBuffer,            // указатель на буфер данных  
    DWORD     nNumberOfBytesToRead, // количество читаемых байтов  
    LPDWORD   lpNumberOfBytesRead,  // количество прочитанных байтов  
    LPOVERLAPPED lpOverlapped      // используется при асинхронной запис  
);
```

POSIX API

```
#include <unistd.h>  
  
ssize_t pread(int fildes, void *buf, size_t nbyte, off_t offset);  
ssize_t read(int fildes, void *buf, size_t nbyte);
```

Управление файлами – Запись

Windows API

```
BOOL WriteFile(  
    HANDLE    hFile,                // дескриптор файла  
    LPCVOID   lpBuffer,             // указатель на буфер данных  
    DWORD     nNumberOfBytesToWrite, // количество записываемых байтов  
    LPDWORD   lpNumberOfBytesWritten, // количество записанных байтов  
    LPOVERLAPPED lpOverlapped       // используется при асинхронной записи  
);
```

POSIX API

```
#include <unistd.h>  
  
ssize_t pwrite(int fildes, const void *buf, size_t nbyte,  
               off_t offset);  
ssize_t write(int fildes, const void *buf, size_t nbyte);
```

Управление файлами – Указатель

Windows API

```
BOOL SetFilePointerEx(  
    HANDLE hFile,                // дескриптор файла  
    LARGE_INTEGER liDistanceToMove, // сдвиг в байтах  
    PLARGE_INTEGER lpNewFilePointer, // новый указатель позиции файла  
    DWORD dwMoveMethod           // начальная точка сдвига  
);
```

POSIX API

```
#include <unistd.h>  
  
off_t lseek(int fildev, off_t offset, int whence);
```

Управление файлами – Атрибуты

Windows API

```
DWORD  GetFileAttributes(  
    LPCTSTR  lpFileName    // имя файла  
);
```

```
BOOL  SetFileAttributes(  
    LPCTSTR  lpFileName,    // имя файла  
    DWORD    dwFileAttributes // атрибуты файла  
);
```

POSIX API

```
[OH] #include <fcntl.h>  
#include <sys/stat.h>  
  
int fstatat(int fd, const char *restrict path,  
            struct stat *restrict buf, int flag);  
int lstat(const char *restrict path, struct stat *restrict buf);  
int stat(const char *restrict path, struct stat *restrict buf);
```

Управление файлами – Блокирование

Windows API

```
BOOL LockFile(  
    HANDLE hFile,  
    DWORD dwFileOffsetLow,  
    DWORD dwFileOffsetHigh,  
    DWORD nNumberOfBytesToLockLow,  
    DWORD nNumberOfBytesToLockHigh  
);
```

```
BOOL UnlockFile(  
    HANDLE hFile,           // дескриптор файла  
    DWORD dwFileOffsetLow, // младшая часть смещения  
    DWORD dwFileOffsetHigh, // старшая часть смещения  
    DWORD nNumberOfBytesToLockLow, // младшая часть количества байтов  
    DWORD nNumberOfBytesToLockHigh // старшая часть количества байтов  
);
```

POSIX API

```
#include <fcntl.h>  
  
int fcntl(int fildes, int cmd, ...);
```

Управление файлами – Сброс буфера

Windows API

```
BOOL FlushFileBuffers(  
    HANDLE hFile    // дескриптор файла  
);
```

POSIX API

```
[FSC]  #include <unistd.h>  
  
int fsync(int fildes); 
```

Управление файлами – WinAPI

Копирование файла

```
BOOL CopyFile(  
    LPCTSTR lpExistingFileName,    // имя существующего файла  
    LPCTSTR lpNewFileName,         // имя нового файла  
    BOOL    bFailIfExists         // действия в случае существования файла  
);
```

Перемещение файла

По сути можно
использовать для
переименования файла

```
BOOL MoveFile(  
    LPCTSTR lpExistingFileName,    // имя существующего файла  
    LPCTSTR lpNewFileName         // имя нового файла  
);
```

Управление файлами – WinAPI

Замещение файла

```
BOOL ReplaceFile(  
    LPCTSTR lpReplacedFileName,    // имя замещаемого файла  
    LPCTSTR lpReplacementFileName, // имя файла-заместителя  
    LPCTSTR lpBackupFileName,      // имя резервной копии файла-заместителя  
    DWORD   dwReplaceFlags,        // опции замещения  
    LPVOID  lpExclude,             // не используется  
    LPVOID  lpReserved             // не используется  
);
```

Информация о файле

```
BOOL GetFileInformationByHandle(  
    HANDLE hFile,    // дескриптор файла  
    // указатель на информацию  
    LPBY_HANDLE_FILE_INFORMATION lpFileInformation  
);
```

Управление файлами – Создание каталога

Windows API

```
BOOL CreateDirectory(  
    LPCTSTR lpPathName,      // имя каталога  
    LPSECURITY_ATTRIBUTES lpSecurityAttributes // атрибуты защиты  
);
```

POSIX API

```
#include <sys/stat.h>  
  
int mkdir(const char *path, mode_t mode);  
  
[OH] ☒#include <fcntl.h>☒  
  
int mkdirat(int fd, const char *path, mode_t mode);
```

Управление файлами – Удаление каталога

Windows API

```
BOOL RemoveDirectory(  
    LPCTSTR lpPathName    // имя каталога  
);
```

POSIX API

```
#include <unistd.h>  
  
int rmdir(const char *path);
```

Управление файлами – Перемещение каталога

Windows API (ограничение: только в рамках одного тома)

```
BOOL MoveFile(  
    LPCTSTR lpExistingFileName,    // имя существующего файла  
    LPCTSTR lpNewFileName          // имя нового файла  
);
```

POSIX API – стандартная функция отсутствует

Управление файлами – Наблюдение за каталогом

Windows API

```
HANDLE FindFirstChangeNotification(  
    LPCTSTR lpPathName,      // имя каталога  
    BOOL bWatchSubtree,      // опция наблюдения  
    DWORD dwNotifyFilter     // условия фильтра  
);
```

```
BOOL FindNextChangeNotification(  
    HANDLE hChangeHandle     // дескриптор для наблюдения за изменениями  
);
```

```
BOOL FindCloseChangeNotification(  
    HANDLE hChangeHandle     // дескриптор для наблюдения за изменениями  
);
```

Управление файлами – Наблюдение за каталогом

Linux API (not POSIX)

```
#include <sys/inotify.h>
```

```
int inotify_init(void);
```

```
#include <sys/inotify.h>
```

```
int inotify_add_watch(int fd, const char *pathname, uint32_t mask);
```

```
#include <sys/inotify.h>
```

```
int inotify_rm_watch(int fd, uint32_t wd);
```

Операционные системы

Лекция 17

Операции над файлами